

A Journey from Acoustic Signals to Web Browsing

Shunjie Zhang [zhangshj2023@shanghaitech.edu.cn]

Kaichen Li [likch2022@shanghaitech.edu.cn]

0. Abstract

This report summarizes the design, implementation, and debugging journey of the CS120 Computer Networks course projects. Over the course of four distinct phases, our group constructed a full network protocol stack—dubbed "Aethernet"—capable of transmitting digital information via acoustic signals and analog cables. The project began with the physical layer (Project 1), moving through the data link layer (Project 2), the network layer (Project 3), and finally the application layer (Project 4). Our experience was defined not just by algorithmic implementation, but by a rigorous battle with hardware constraints and operating system configurations. While our software logic eventually matured to support IP fragmentation, NAT, and HTTP requests, our most significant lessons were learned in the trenches of the physical and link layers. We discovered that in a software-defined radio (SDR) environment using consumer-grade audio equipment, code correctness is only half the battle. This report details our transition from a struggling acoustic link plagued by ASIO misconfigurations to a stable, web-browsing network node, highlighting the critical role of hardware environment management.

1. Introduction

The objective of the CS120 course projects was to demystify the abstraction layers of the TCP/IP stack by building them from scratch. Unlike standard socket programming assignments where the lower layers are taken for granted, we were tasked with modulating bits into sound waves, handling collisions on a shared medium, and routing packets across distinct subnets.

The architecture, "Aethernet," relies on standard computer audio interfaces (microphones and speakers/audio cables) to simulate the physical transmission medium. The ultimate goal was to allow a computer (Node 1) to browse the real internet through a gateway (Node 2) using this custom stack. This required a rigorous implementation of modulation (PSK/OFDM), framing, error detection (CRC), Automatic Repeat Request (ARQ), Carrier Sense Multiple Access (CSMA), IP forwarding, and higher-level protocols like DNS and HTTP.

2. Project 1: The Acoustic Link (The Physical Layer)

The foundation of any network is the Physical Layer (PHY). In Project 1, our goal was to establish a reliable link between two air-gapped computers using sound waves.

2.1 Technical Implementation Details

We implemented **Phase Shift Keying (PSK)** as our primary modulation scheme. To maximize data throughput and separate channels, we utilized **Orthogonal Frequency-Division Multiplexing (OFDM)**, specifically selecting carrier frequencies at **2000 Hz and 10000 Hz**. Additionally, to ensure transmission reliability in the noisy acoustic environment, we integrated **Hamming Codes** for Forward Error Correction (FEC).

2.2 The Struggle with Reliability

Initially, our progress was stalled by severe reliability issues. Despite implementing robust frame synchronization using chirp preambles, our Bit Error Rate (BER) remained unacceptably high. We observed that the received signal was often distorted or suffered from unpredictable time-drifts that our synchronization logic could not handle.

We spent days debugging the demodulation logic, assuming the error lay in our mathematical implementation of the convolution or the decision boundaries for the constellation diagram. In a moment of desperation, assuming our codebase was fundamentally flawed, we decided to scrap our initial implementation entirely. We even attempted to switch programming languages, migrating our prototype logic to see if performance issues were language-specific.

2.3 The ASIO Breakthrough

The turning point came when we stepped back from the signal processing math and looked at how the software interacted with the hardware. We discovered that our application was not successfully invoking the Audio Stream Input/Output (ASIO) interface.

Without ASIO, our audio data was being routed through the operating system's standard audio mixer (e.g., DirectSound or WASAPI in shared mode). This introduced significant latency, jitter, and, most critically, implicit resampling. The OS was resampling our carefully crafted 48kHz signals, introducing aliasing artifacts and noise that ruined the orthogonality of our symbols.

Once we fixed the code to correctly initialize the ASIO driver, the difference was night and day. We achieved direct, low-latency access to the sound card's DAC and ADC. The signal-to-noise ratio improved immediately, and our demodulation algorithms began working as intended. This experience taught us that in software-defined communication, the interface between the software and the kernel driver is the most fragile point of failure.

3. Project 2: Managing Multiple Access (The MAC Layer)

With a functional physical layer (transitioning from air-gapped sound to audio cables for stability), Project 2 focused on the Data Link Layer. We needed to manage how multiple nodes shared the same physical medium without corrupting each other's transmissions.

3.1 Technical Implementation Details

Given the transition to wired audio transmission, we switched our modulation strategy to **Line Coding**. To ensure data integrity, we implemented **CRC checksums** and a strict **ACK** (Acknowledgement) mechanism. Furthermore, to handle shared medium contention, we implemented **Carrier Sense Multiple Access (CSMA)**. Before sending any data packet, the system listens to see if the channel is idle. If the channel is busy or a collision occurs, we employ a **Binary Exponential Backoff** algorithm to wait for a random period before retrying. This strategy proved highly effective, allowing us to successfully pass the **Task 4 Jammer interference test**.

3.2 The Collision Crisis

The core challenge of this project was implementing a reliable ARQ (Automatic Repeat reQuest) system combined with CSMA. In our initial testing, we encountered a persistent "Link Error." The scenario was as follows: Node A would send a Data frame. Node B would receive it and attempt to send an ACK frame immediately. However, due to propagation delays and processing latency, Node A might mistakenly believe the channel was free or timeout prematurely and attempt to retransmit the Data frame exactly when Node B was sending the ACK. These collisions led to corrupted frames, repeated timeouts, and eventual link failure.

3.3 The Hardware Variable

However, even with the backoff algorithm, we still faced inexplicable packet drops. This forced us to look beyond the code once again. We conducted a systematic review of our physical setup and identified two major hardware-related culprits:

1. **Volume Imbalance:** We discovered that the output volume of the sending node and the input gain of the receiving node were not perfectly matched. If the sender was too loud, it caused clipping (saturation) in the receiver's ADC, making the waveform unrecognizable. If it was too quiet, the signal was lost in the noise floor. Calibrating the volumes to a "sweet spot" was essential.
2. **Bluetooth Interference:** We noticed that when one of the group member's laptops had Bluetooth enabled, the transmission reliability dropped, even though we were using wired audio cables. We hypothesize that the Bluetooth module, sharing the same 2.4GHz spectrum as some internal bus lines or perhaps causing CPU interrupt spikes, was interfering with the real-time processing

required by the ASIO buffer. Turning off Bluetooth on all test machines immediately improved stability.

These findings reinforced the lesson from Project 1: environmental variables are just as critical as the code itself.

4. Project 3: To the Internet (The Network Layer)

Project 3 tasked us with implementing the Internet Protocol (IP), ICMP, and routing logic. This included assigning IP addresses, handling Subnets, and implementing Network Address Translation (NAT) to allow our private "Aethernet" nodes to communicate with the public Internet via a gateway.

4.1 Technical Implementation Details

We implemented the logic to encapsulate IP headers and forward **ICMP requests** to the external world. This allowed an offline computer to connect to an online node via audio cable and access the external network. To ensure that the **Ping latency** remained within a reasonable range, we optimized the transmission strategy: instead of sending the full, heavy packet content over the audio link, we chose to send **custom, minimal ICMP requests**. This optimization significantly reduced transmission time while maintaining functionality.

4.2 Smooth Sailing

Surprisingly, Project 3 was the smoothest phase of the entire semester. This success was not due to the simplicity of the task—implementing a NAT table and recalculating IP checksums is non-trivial—but rather due to the stability of our lower layers.

Because we had rigorously debugged the ASIO interface in Project 1 and solved the collision/volume issues in Project 2, the underlying transport was reliable. The abstraction provided by the layered network model worked exactly as intended. We could focus entirely on packet logic (TTL decrements, routing table lookups, and ICMP Echo replies) without worrying about whether a bit was flipped due to audio clipping.

4.3 Integration

We successfully configured Node 2 as a router. It accepted Aethernet frames, stripped the headers, analyzed the IP packet, and forwarded it to the campus Wi-Fi interface using standard socket operations (simulating the "WAN" link). We successfully pinged `8.8.8.8` from a node connected only via audio cable, a milestone that validated our entire stack design.

5. Project 4: Above IP (The Application Layer)

The final phase involved implementing the Application Layer, specifically a DNS client and a simplified HTTP client capable of a TCP handshake.

5.1 Technical Implementation Details

For the DNS requirement, we first utilized **Wireshark** to determine the correct structure of DNS packets. We then implemented a **recursive query** mechanism to resolve domain names to their belonging IP addresses. In our HTTP web scraping implementation, we simulated the **TCP protocol** by manually handling the three-way handshake and confirming sequence numbers. We also simulated a **Sliding Window** mechanism to reliably capture and reassemble the HTML content sent by the remote server.

5.2 Final Implementation

Like Project 3, this phase was relatively straightforward due to our solid foundation. We implemented a raw TCP socket over our Aethernet IP stack. We constructed the TCP SYN packet manually, sent it to a public web server (e.g., `example.com`), waited for the SYN-ACK, and responded with ACK. Once the connection was established, we crafted a raw HTTP GET request. Seeing the HTML code of a real website return through our audio cables and appear in our terminal was the highlight of the course. It proved that our "toy" network was mathematically and logically compatible with the massive infrastructure of the World Wide Web.

6. Discussion: The Hardware Reality

While our code eventually worked, the most valuable knowledge we gained from CS120 was not about checksums or window sizes, but about the "Physical" reality of the Physical Layer. If we were to advise future students or write a "survival guide" for this course, it would focus heavily on the hardware ecosystem.

6.1 The "Intermittent Normal"

One of the most frustrating aspects of this project was the phenomenon of "Intermittent Normality." A setup would work perfectly during a Tuesday night test, achieving 100% throughput. On Wednesday morning, without a single line of code changing, the same setup would fail.

We learned to treat the hardware with suspicion. Audio cables are prone to internal micro-fractures that only manifest when the cable is twisted a certain way. The 3.5mm jacks on USB sound cards can become loose, causing channel cross-talk or ground loops. We eventually adopted a protocol of

"hardware sanity checks" before every coding session: swapping cables, cleaning jacks, and verifying connections.

6.2 The Checklist of "Hidden" Settings

Through our failures in Project 1 and 2, we compiled a checklist of non-code factors that are fatal to the Aethernet:

1. **ASIO Consistency:** It is not enough to just "use ASIO." The buffer size and sample rate settings must be identical across all nodes and the driver control panel. A mismatch here leads to drift that no amount of software synchronization can fix.
2. **Volume Management:** "Louder is better" is a fallacy. We found that 80-90% volume is usually the limit before non-linear harmonic distortion sets in. Conversely, software gain must be balanced against the noise floor.
3. **Bluetooth Interference:** It is crucial to **ensure Bluetooth is disabled** when using audio cables. We found that active Bluetooth can prevent the ASIO driver from successfully connecting to the USB interface.
4. **Input/Output Selection:** Windows and Linux often try to be "smart" by switching default audio devices when a new jack is plugged in. Hardcoding the device ID in the ASIO wrapper is safer than relying on "Default Device."

6.3 Hardware vs. Software

In Computer Science, we are often taught that if the logic is correct, the program will work. This course challenged that notion. We learned that ****Software Defined Radio is 50% Software and 50% Radio.**** No amount of error-correcting code (Reed-Solomon or CRC) can compensate for a physical medium that is fundamentally broken due to a bad cable or a misconfigured driver.

7. Conclusion

The development of the Aethernet stack was a journey of oscillation between despair and triumph. Project 1 and 2 were the "filters"—the phases that forced us to confront the harsh reality of analog signal processing and shared-medium contention. By overcoming the ASIO configuration errors and the physical interference issues, we earned the stability that made Project 3 and 4 enjoyable explorations of logical protocols.

Our final implementation stands as a functional, albeit slow, bridge to the digital world. We successfully tunneled the complex protocols of the modern internet through the vibration of air (and voltage on audio cables). More importantly, we learned that in systems programming, one must look "full stack"—not just from Application to Physical, but all the way down to the voltage, the driver, and the cable.